



MODULE 9

Testing & Debugging for iOS

Enterprise Mobile Application Development Bootcamp

Phase 02 | iOS Development with Swift

Module 9 Roadmap

Six blocks. By the end: a fully tested, debugged PNC Mobile feature, ready for the capstone.



The Testing Pyramid & XCTest

- Why Test at All
- The Testing Pyramid
- XCTest & the AAA Pattern
- Live TDD Mini-Demo



The iOS Testing Pyramid

UI Tests

3–10 tests covering critical paths

Integration Tests

Repository + real Core Data, mocked network

Unit Tests

ViewModel · Service · Model - fast, isolated, many

< 1ms

Typical unit test run time

30+ sec

Typical UI test run time

70%+

Target ViewModel/Service coverage



Using Swift Testing

- Swift Testing is the modern approach to unit testing in Swift applications
- Tests can be written in classes, structs, and actors
- Tests are functions that are marked with the @Test wrapper
- The framework has built-in concurrency support

Code Bite

```
import Testing
@testable import YourAppTarget

struct CalculatorTests {

    @Test func testAddition() {
        let calculator = Calculator()
        let result = calculator.add(5, 3)
        #expect(result == 8)
    }
}
```



@testable

- Importing a module typically gives your code access to the public assets from that module
- Module assets declared as internal will be visible only within that module
 - They will not be visible to the code that is importing the module
- Occasionally you may need to access those internal assets during unit tests
- Adding the @testable wrapper to the import will grant access to those internal assets
 - Provided the module was compiled with the enable-testing flag
 - Xcode will handle this automatically for debug and test builds
- This should be used sparingly
 - Tests should primarily focus on the public interface of components



The AAA Pattern

- Tests frequently need three phases
 - Setup initial conditions
 - Execute the code under test
 - Verify the results
- Those steps are often called:
 - Arrange
 - Act
 - Assert
- Returning a Boolean success value from a test would be too limiting
 - Why did the test fail?
 - How did it fail?

Code Bite

```
// Arrange
let mock = MyMockService()
let comp = SomeComponent(svc: mock)

// Act
let result = comp.performAction()

// Assert
#expect(result == 4)
```



Assertions

- Assertions are statements about what should be the case after the code under test executes
- `#expect` is a built-in macro that checks whether a condition or expression is true
- It features soft assertions
 - A failure is recorded but it doesn't stop the test execution
- It features expression capture
 - Expression values are stored and output so you can better see why it failed
- `#expect(throws:)` can verify that a error is thrown

Code Bite

```
#expect(result == true)

#expect(total > 0)

#expect(!name.isEmpty,
        "name should always be supplied")

#expect(throws: ValidationError.tooSmall)
```



XCTest: the Legacy Approach

- XCTest is the older testing module for Swift
- Classes that contain unit tests must inherit from XCTestCase
- Every method in those classes whose name starts with lowercase "test" will be executed as unit tests
- The XCTAssertEqual function is executed to make assertions
- This framework does NOT support concurrency out of the box
 - It must be configured, if desired

Code Bite

```
class TransferServiceTests: XCTestCase {  
    func test_transferBelowBalance_succeeds() {  
        let fakeRepo = FakeAccountsRepository()  
        let viewModel = TransferViewModel(  
                                repository: fakeRepo)  
        let account = Account(id: "1",  
                                name: "Checking",  
                                maskedNumber: "0000", balance: 500)  
        await viewModel.attemptTransfer(amount: 100,  
                                        from: account, to: account)  
        XCTAssertEqual(fakeRepo.transferCallCount, 1)  
    }  
}
```



Live TDD Mini-Demo: Red, Then Green

- 1 Write the test FIRST, for a function that doesn't exist yet - it won't even compile
- 2 Write just enough of a stub to make it compile, then run it - RED (test fails)
- 3 Write the minimum implementation needed to make the assertion true
- 4 Run again - GREEN (test passes)
- 5 Refactor with confidence - the test now guards against regressions



Block 1 - The Testing Pyramid and Swift Testing

Skill Check ✓

- Why is the testing pyramid bottom-heavy - why not write mostly UI tests since they test the most realistic scenario?
- Name the three sections of the AAA pattern in order.
- In TDD, what does 'red' mean, and what should you do immediately after seeing it?



Async Testing & Test Doubles

- Testing async/await Code
- Fakes, Stubs & Mocks
- When Each Type Is Appropriate



Testing async/await Code

```
@Test func loadAccountsPopulatesViewModelFromRepository() async {  
    let fake = FakeAccountsRepository()  
    fake.stubbedAccounts = [Account(id: "1", name: "Checking", maskedNumber: "0000",  
        balance: 500)]  
    let viewModel = AccountsViewModel(repository: fake)  
  
    await viewModel.loadAccounts()    // just await it, like any other call  
  
    #expect(viewModel.accounts.count, 1)  
}  
  
// No special async test framework needed - Swift Testing supports async test  
// methods natively.
```



Fakes, Stubs & Mocks: the Vocabulary



Fake

A working, simplified implementation - e.g. FakeAccountsRepository just returns an in-memory array instead of hitting a real API



Stub

Returns canned, hard-coded answers to specific calls - no real logic behind it at all



Mock/Spy

Like a stub, but also records HOW it was called, so a test can assert on interactions (e.g. 'was transfer() called exactly once?')



A Mock Class

```
final class MockAccountsRepository: AccountsRepository {  
    var stubbedAccounts: [Account] = []  
    private(set) var fetchAccountsCallCount = 0    // <- the "mock" part  
  
    func fetchAccounts() async throws -> [Account] {  
        fetchAccountsCallCount += 1  
        return stubbedAccounts  
    }  
}
```



Using a Mock in Tests

```
@Test func loadAccountsCallsRepositoryExactlyOnce() async {  
    let mock = MockAccountsRepository()  
    let viewModel = AccountsViewModel(repository: mock)  
  
    await viewModel.loadAccounts()  
  
    #expect(mock.fetchAccountsCallCount, 1)    // asserting on the INTERACTION  
}
```



Block 2 - Async Testing & Test Doubles

Skill Check ✓

- What's the one syntactic difference between testing async code and testing synchronous code in XCTest?
- What's the difference between a stub and a mock?
- Which category does the FakeAccountsRepository from Module 7/8 belong to - fake, stub, or mock?



UI Testing

- XCTest Fundamentals
- Accessibility Identifiers
- Recording & Writing UI Tests



UI Tests

- Swift Testing was introduced in Xcode 16
- It is the modern choice for Unit Tests
- However, UI automation in Xcode still relies on XCTest



Setting Up a UI Test

- UI tests must be defined in a class inheriting from XCTestCase
- Override setUpWithError to start the application anew before each test
- Override tearDownWithError to clean up after each test
- Setting continueAfterFailure to false will stop executing remaining tests if one fails

Code Bite

```
final class CounterUITests: XCTestCase {  
    private var app: XCUIApplication!  
  
    override func setUpWithError() throws {  
        continueAfterFailure = false  
        app = XCUIApplication()  
        app.launch()  
    }  
  
    override func tearDownWithError() throws {  
        app = nil  
    }  
}
```



Accessibility Identifiers: UI Testing's Foundation

- Without an accessibilityIdentifier, UI tests match on visible label text
- This breaks the moment copy changes
 - A localization update or simple wording tweak can silently break a whole test suite
- Identifiers are stable; text is not.

Code Bite

```
TextField("Username", text: $username)
    .accessibilityIdentifier("usernameField")

Button("Log In") { login() }
    .accessibilityIdentifier("loginButton")
```



XCUITest Fundamentals

```
func test_login_navigatesToAccountsList() {  
  
    app.textFields["usernameField"].tap()  
    app.textFields["usernameField"].typeText("demo")  
    app.secureTextFields["passwordField"].tap()  
    app.secureTextFields["passwordField"].typeText("demo1234")  
    app.buttons["loginButton"].tap()  
  
    XCTAssertTrue(app.navigationBars["Accounts"].waitForExistence(timeout: 5))  
}
```



Locating Elements Onscreen

- The XCUIApplication object allows you to find particular kinds of elements
- Collections include:
 - buttons
 - staticTexts
 - textFields
 - secureTextFields
 - collectionViews
 - otherElements
- It is not always logical what kind of element a particular view will be rendered as
- In your test, you could print the `app.debugDescription` to print out the accessibility hierarchy to see what kind of element was rendered



Block 3 - UI Testing

Skill Check ✓

- Per the testing pyramid, roughly how many UI tests should a feature like PNC Mobile's transfer flow have?
- Why use accessibilityIdentifier for UI test targeting instead of matching on visible label text?
- What Xcode tool can generate UI test code automatically from a recorded interaction?



Debugging Techniques

- Breakpoints Beyond the Basics
- LLDB Essentials
- The View Debugger
- A Systematic Approach



Breakpoints Beyond 'Click the Line Number'



Conditional breakpoints

Right-click → Edit Breakpoint → only pause when a condition is true, e.g. `account.balance < 0`



Exception breakpoints

Pause on EVERY thrown error/exception, anywhere in the app - essential for catching a crash's true origin



Symbolic breakpoints

Pause when a specific system function is called, e.g. `UIViewAlertForUnsatisfiableConstraints` (Module 6's Auto Layout bug, made visible)



LLDB Essentials

```
po <expression>
```

Print the description of an object - the one most participants already know

```
p <expression>
```

Print a raw value without invoking description - faster for primitives

```
expr <statement>
```

Execute arbitrary code at the breakpoint - even reassign a variable to test a fix live

```
bt
```

Print the full backtrace - the call stack that led here

expr can reassign a variable at the breakpoint to test a hypothesis about a fix without stopping, editing, and re-running - a significant time saver for hard-to-reproduce bugs.



A Systematic Approach - Recalling Phase 01

- 1 Reproduce the bug reliably before touching any code
- 2 Read the actual error/crash message and stack trace - fully, not just the first line
- 3 Form a specific, falsifiable hypothesis about the cause
- 4 Use breakpoints/LLDB to test that hypothesis directly
- 5 Fix the root cause, not just the symptom
- 6 Write a test that would have caught this bug
- 7 Verify the fix and confirm the new test passes



Block 4 - Debugging Techniques

Skill Check ✓

- What's the difference between a conditional breakpoint and an exception breakpoint?
- What LLDB command lets you print an object's description at a paused breakpoint?
- In the seven-step debugging framework, what comes immediately after forming a hypothesis?



Performance Profiling

- Instruments Fundamentals
- The Time Profiler
- Reading a Flame Graph



Instruments: Xcode's Profiling Suite



Time Profiler

Where is CPU time actually being spent? The most commonly used template



Allocations

What's being allocated, how much memory, and does it ever get freed?



Leaks

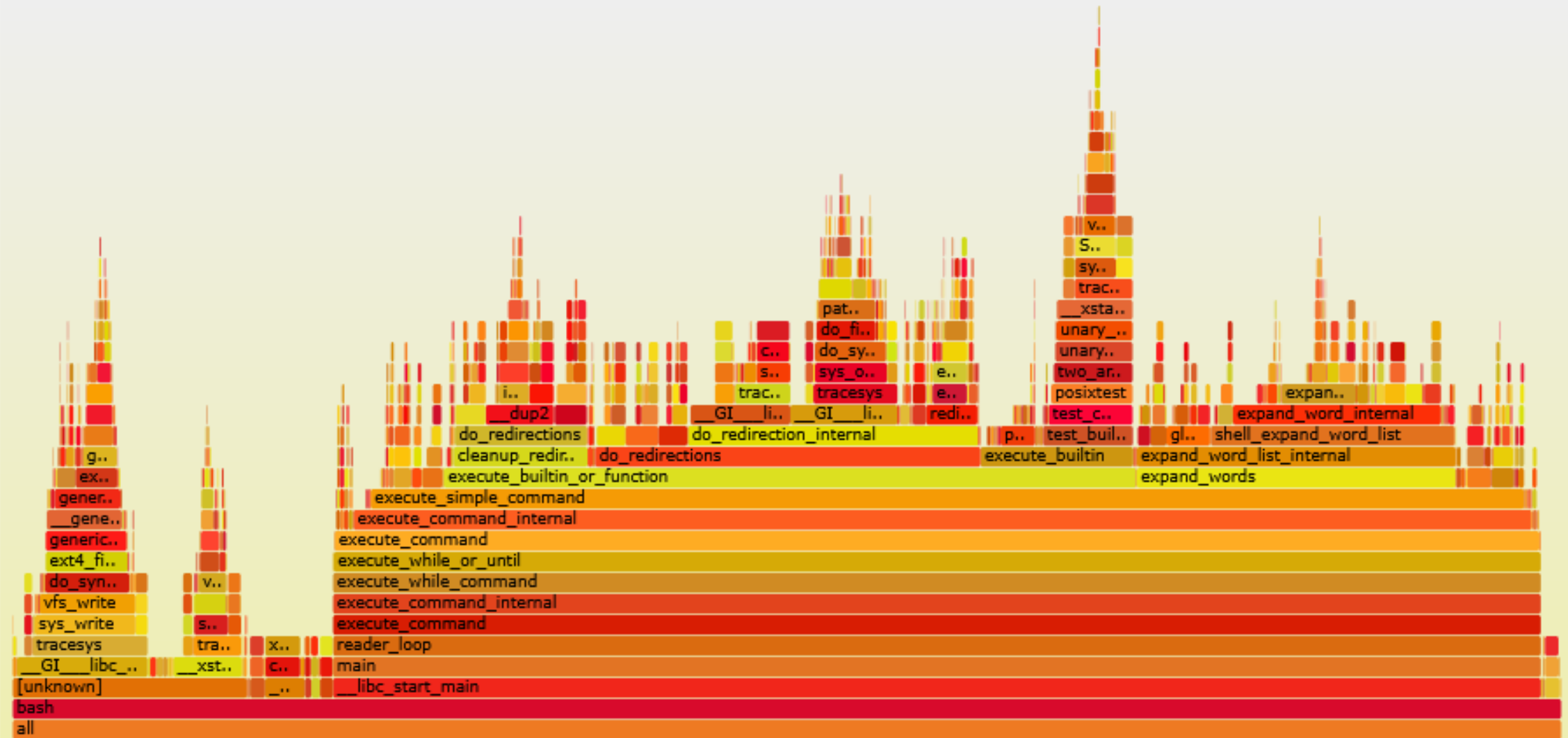
Automatically flags objects that were allocated but never released



Network

Every request/response the app makes, with timing - useful after Module 8





Reading a Flame Graph

Width = time spent in that function.
Height = how deep the call stack goes.
Wide bars towards the top are possible performance bottlenecks.



Block 5 - Performance Profiling

Skill Check ✓

- In a flame graph, what does the WIDTH of a bar represent?
- Which Instruments template would you reach for to answer 'where is my CPU time going'?
- Why is a synchronous JSON decode of a large payload on the main thread a performance concern?



Memory Leaks & Crash Analysis

- Retain Cycles
- [weak self] & [unowned self]
- Reading a Crash Log



Retain Cycles: the Classic iOS Memory Leak

Leaks - strong capture of self

```
class TransferViewModel {  
    var onSuccess: (() -> Void)?  
  
    func attemptTransfer() {  
        onSuccess = {  
            self.showConfirmation()  
            // strong capture - retain cycle  
        }  
    }  
}
```

Fixed - [weak self]

```
class TransferViewModel {  
    var onSuccess: (() -> Void)?  
  
    func attemptTransfer() {  
        onSuccess = { [weak self] in  
            self?.showConfirmation()  
        }  
    }  
}
```

[weak self] vs [unowned self]: weak produces an Optional you must safely unwrap (self? or if let self); unowned assumes self will always still exist and crashes if it doesn't. Default to weak unless you can prove otherwise.

The Leaks profiling instrument can help detect these memory leaks!



Reading a Crash Log



Exception Type

What kind of crash - EXC_BAD_ACCESS (memory), SIGABRT (assertion/fatalError), etc.



Crashed Thread

Skip straight here - this stack trace is almost always where the real answer is



Symbolication

Raw crash logs show memory addresses; symbolication maps them back to actual function/file names - Xcode does this automatically for your own builds

Details and step-by-step instructions are available in the official docs:

<https://developer.apple.com/documentation/xcode/diagnosing-issues-using-crash-reports-and-device-logs>



Block 6 - Memory Leaks & Crash Analysis

Skill Check ✓

- In the retain cycle example, what two objects are holding strong references to each other?
- What's the practical difference between [weak self] and [unowned self]?
- In a crash log, which section should you read first, and why?

